

If a Broken Clock is Right Twice a Day

Arian Nazarian



Figure 1: Damaged Omega Speedmaster 3221.30 — forever at 10:08:00 on a 12-hour dial on Saturday, March 11⁴.

Abstract

What is the probability of randomly checking a broken time-only 12-hour watch and discovering that it matches exactly with the current local time? If the dial has no AM/PM indicator, the calculation is a simple finite-counting problem: two matching instants occur in each 24-hour day at any chosen level of time precision. What if the watch is an Omega Speedmaster 3221.30 "Day-Date," displaying the time, month, weekday, and date? How often does such a configuration align with reality and what is the probability of a perfect match when checked randomly?

These seemingly trivial questions may suggest inconsequential solutions to the uninitiated, but consider a traveler that is moored in time, bound not by will but by a watch standing still. The hands are stuck, the calendar unchanged, yet this broken mechanism serves as their only guide. They may only journey to the rare intersections where time's machinery meets its motionless twin.

The irregular yet cyclical structure of the Gregorian calendar presents a finite but non-uniform recurrence problem. This paper explores the calendar's underlying mathematical framework, compares algorithmic approaches to computing date recurrences, and gives exact probabilities for when a fixed date-time display reoccurs. The main calculation is carried out over the Gregorian 400-year cycle, whose 146,097 days form the natural sample space for the problem.

1 The Gregorian Calendar

The Gregorian calendar was adopted in 1582 as a reform of the Julian calendar.^{1;6} Its date representations are standardized in ISO 8601.³

The Julian system assumed

$$365.25 \text{ days,}$$

whereas the actual tropical year is approximately

$$365.2422 \text{ days,}$$

so the Julian calendar drifted by

$$365.25 - 365.2422 \approx 0.0078 \text{ days/year.}$$

Over the centuries this small discrepancy accumulated to roughly 10 days, displacing the spring equinox from its traditional date around 21 March. The 1582 reform therefore omitted 10 calendar days to realign the seasons.

In addition to this single adjustment, the Gregorian calendar introduced a modified leap-year rule to minimize future drift:^{1;6}

- every year divisible by 4 is a leap year;
- except years divisible by 100, which are not leap years;
- unless they are also divisible by 400, in which case they remain leap years.

This rule yields 97 leap years in each 400-year cycle and an average year length of

$$\frac{(303 \times 365) + (97 \times 366)}{400} = 365.2425 \text{ days,}^1$$

an error of only about one day every 3 300 years. Because a 400-year block contains exactly 146 097 days, it serves as the fundamental period for many calendar algorithms, including Zeller's congruence.^{1;6}

2 Zeller's Congruence: Computing the Day of the Week

Zeller's congruence is an efficient algorithm for determining the weekday of any Gregorian date.⁷ It is expressed as

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7, \quad (1)$$

where

- h is the weekday (0 = Saturday, 1 = Sunday, ..., 6 = Friday);
- q is the day of the month;
- m is the *modified* month: March = 3, ..., December = 12, January = 13, February = 14 (treated as months of the previous year);
- $K = Y \bmod 100$ (year within the century);
- $J = \lfloor Y/100 \rfloor$ (century).

Handling January and February

Because January and February are counted as months 13 and 14 of the *previous* year, the calendar year must be decremented by one whenever $m > 12$:

$$\text{if } m > 12, \quad Y \leftarrow Y - 1.$$

This shift automatically aligns leap-year logic and century boundaries.

Alternative Formulation: Starting Months at One

If months use the conventional numbering (January = 1, ..., December = 12), let M be that month index. Zeller's formula becomes⁷

$$h = \left(q + \left\lfloor \frac{13(M+1+12L)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7,$$

where

$$L = \begin{cases} 1, & M \leq 2 \text{ (January or February)}, \\ 0, & \text{otherwise,} \end{cases} \quad K = (Y - L) \bmod 100, \quad J = \lfloor (Y - L)/100 \rfloor.$$

The adjustment term L explicitly handles the leap-year boundary, albeit at the cost of a slightly more complex expression.

3 Direct Derivation from Gregorian Rules

An alternative approach derives a formula by directly following the Gregorian calendar's definitions. This method computes the total number of days elapsed since a fixed reference date (typically January 1 of year 1 in the proleptic Gregorian calendar) and then determines the day of the week by taking the result modulo 7.^{1:6}

Constructing the Formula

Using q to denote the day of the month, the weekday can be computed by counting all elapsed days before the date and reducing modulo 7:

$$h = (h_0 + (q - 1) + T(Y) + M_{\text{offset}}(M, Y)) \bmod 7,$$

where:

- h represents the day of the week under the chosen numbering convention.
- h_0 is the weekday of 1 January in year 1 under that convention. With Zeller's convention $0 = \text{Saturday}$, $1 = \text{Sunday}$, ..., $6 = \text{Friday}$, one has $h_0 = 2$, since 1 January 1 is Monday in the proleptic Gregorian calendar.
- $T(Y)$ represents the total number of days from all full years before Y .
- $M_{\text{offset}}(M, Y)$ represents the total number of days from completed months in year Y .

Counting Days from Full Years

The total number of days contributed by past years before year Y is:

$$T(Y) = (Y - 1) \times 365 + C(Y - 1).$$

Here, $C(n)$ counts the leap years among years $1, 2, \dots, n$:

$$C(n) = \lfloor n/4 \rfloor - \lfloor n/100 \rfloor + \lfloor n/400 \rfloor.$$

For the current year itself, it is useful to separate the cumulative count $C(n)$ from the leap-year indicator

$$\ell(Y) = \begin{cases} 1, & 4 \mid Y \text{ and } (100 \nmid Y \text{ or } 400 \mid Y), \\ 0, & \text{otherwise.} \end{cases}$$

Accumulating Days from Completed Months

The cumulative number of days contributed by past months in year Y is given by:

$$M_{\text{offset}}(M, Y) = \begin{cases} 0, & M = 1 \text{ (January)} \\ 31, & M = 2 \text{ (February)} \\ 31 + 28 + \ell(Y), & M = 3 \text{ (March)} \\ 31 + 28 + \ell(Y) + 31, & M = 4 \text{ (April)} \\ 31 + 28 + \ell(Y) + 31 + 30, & M = 5 \text{ (May)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31, & M = 6 \text{ (June)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30, & M = 7 \text{ (July)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31, & M = 8 \text{ (August)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31, & M = 9 \text{ (September)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30, & M = 10 \text{ (October)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31, & M = 11 \text{ (November)} \\ 31 + 28 + \ell(Y) + 31 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 30, & M = 12 \text{ (December)} \end{cases}$$

For months beyond February, the function accounts for leap years by adding the indicator $\ell(Y)$, not the cumulative leap-year count $C(Y)$. Although the piecewise definition of $M_{\text{offset}}(M, Y)$ appears unwieldy, it clearly enumerates the cumulative days leading up to each month.

Why Modulo 7 Determines the Weekday

Since the formula ultimately relies on taking the result modulo 7, let us examine why this step works:

- The sequence of weekdays follows a repeating cycle every 7 days.
- A normal year contains 365 days, which shifts the weekday forward by $365 \bmod 7 = 1$.
- A leap year contains 366 days, shifting the weekday forward by $366 \bmod 7 = 2$.
- The leap-year indicator $\ell(Y)$ adjusts for the extra day in leap years, preventing long-term drift.

Thus, by summing the total days and computing modulo 7, we effectively “wrap around” to obtain the correct weekday.

Comparison with Zeller’s Congruence

While both this direct derivation and Zeller’s congruence ultimately determine the day of the week, they differ in approach and computational efficiency.

- The direct derivation explicitly follows the Gregorian calendar’s structure, computing total elapsed days from a fixed reference date. It provides a conceptual and systematic understanding of how weekdays cycle over time but requires separate handling of year, month, and day contributions.
- Zeller’s congruence, on the other hand, is a more compact formula that applies modular arithmetic directly to a given date without explicitly summing total elapsed days. It incorporates a pre-adjusted month numbering system and operates in a form that is computationally efficient for quick lookups.
- The direct derivation is particularly useful for historical and educational purposes, as it breaks down the logic step by step, whereas Zeller’s congruence is optimized for direct calculation in programming and algorithmic implementations.

This derivation follows the structure of the Gregorian calendar directly, ensuring an intuitive calculation of the weekday for any given date. It highlights how ordinary years advance the weekday by one, leap years advance it by two, and the Gregorian correction terms preserve a repeating structure modulo 7—a fundamental aspect also utilized elegantly in Zeller’s congruence.^{1:6}

4 Manipulating Zeller's Congruence to Find Matching Years

Zeller's congruence is typically used to compute the day of the week for a given date. However, it can also be rearranged to find all years within a specified range in which a particular weekday, month, and date coincide.⁷

Rewriting the Formula

The standard form of Zeller's congruence for a date (Y, M, q) is:

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7,$$

where

- h is the day of the week (0 = Saturday, 1 = Sunday, ..., 6 = Friday);
- q is the day of the month;
- m is the modified month (March = 3, ..., December = 12; January = 13, February = 14 of the previous year);
- $K = Y \bmod 100$;
- $J = \lfloor Y/100 \rfloor$.

Given a target weekday h_{target} , we seek all years Y that satisfy

$$h_{\text{target}} = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \right) \bmod 7.$$

Isolating Year Components

Expressing K and J explicitly as functions of Y :

$$K = Y \bmod 100, \quad J = \lfloor Y/100 \rfloor,$$

and substituting gives

$$h_{\text{target}} = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + (Y \bmod 100) + \left\lfloor \frac{Y \bmod 100}{4} \right\rfloor + \left\lfloor \frac{\lfloor Y/100 \rfloor}{4} \right\rfloor - 2\lfloor Y/100 \rfloor \right) \bmod 7,$$

which rearranges to the congruence

$$(Y \bmod 100) + \left\lfloor \frac{Y \bmod 100}{4} \right\rfloor + \left\lfloor \frac{\lfloor Y/100 \rfloor}{4} \right\rfloor - 2\lfloor Y/100 \rfloor \equiv h_{\text{target}} - q - \left\lfloor \frac{13(m+1)}{5} \right\rfloor \pmod{7}.$$

Algorithm for Finding Matching Years

Rather than solving this congruence algebraically, a practical method iterates over a range of years and selects those that satisfy the condition. The 400-year Gregorian cycle is short enough that this is exact, transparent, and computationally negligible. A straightforward algorithm is therefore

1. iterate through a specified range of years;
2. for each year compute h via Zeller's congruence;
3. if $h = h_{\text{target}}$, record the year.

This iterative approach efficiently returns every year in which the chosen weekday, month, and date coincide.

5 Comparison of Algorithm Implementations

This section contrasts two approaches for locating every year in which a given weekday, month, and date coincide in the Gregorian calendar: a Python routine based on Zeller’s congruence⁷ and a JavaScript routine that relies on the built-in `Date` object defined by the ECMAScript specification.²

Feature	Python (Zeller)	JavaScript (UTC Date)
Month numbering	March = 3, ..., December = 12; Jan = 13, Feb = 14 (prev. year)	Jan = 0, ..., December = 11 (ECMA-262)
Weekday numbering	0 = Saturday, ..., 6 = Friday	0 = Sunday, ..., 6 = Saturday (ECMA-262)
Leap-year handling	Implicit in Zeller’s arithmetic	Engine follows Gregorian rule
Century rollover	Handled via Jan/Feb shift	Built into <code>Date</code> arithmetic
Computation style	Pure modular arithmetic	UTC date ops
Use case	Direct offline scans	Browser-based date lookups

Table 1: Python implementation of Zeller’s congruence vs. ECMAScript `Date`.

Both methods correctly identify, for example, every year in which 11 March falls on a Saturday.

Python Script with Zeller’s Congruence

The listing below applies Zeller’s congruence (using months 3–14) to scan a year range.

Listing 1: Python implementation of Zeller’s congruence for matching years

```
def find_matching_years(target_weekday, month, day, start_year, end_year):
    def zellers_congruence(day, month, year):
        if month < 3:
            month += 12 # January becomes 13, February becomes 14
            year -= 1 # Treat January and February as months of the previous year

        K = year % 100 # Year within the century
        J = year // 100 # Century number

        h = (day + (13 * (month + 1)) // 5 + K + (K // 4) + (J // 4) - 2 * J) % 7

        return h # 0=Saturday, 1=Sunday, ..., 6=Friday

    matching_years = []
    for year in range(start_year, end_year + 1):
        if zellers_congruence(day, month, year) == target_weekday:
            matching_years.append(year)

    return matching_years

# Find years where March 11 is a Saturday
print(find_matching_years(0, 3, 11, 2001, 2100))

# Output: [2006, 2017, 2023, 2028, 2034, 2045, 2051, 2056, 2062, 2073, 2079, 2084, 2090]
```

- January and February are treated as months 13 and 14 of the previous year.
- Century and leap-year corrections are implicit.
- The script iterates through a range and returns all years whose weekday matches.

JavaScript Using the Date Object

The ECMAScript `Date` object provides leap-year and century handling out of the box. The implementation below uses UTC methods so that the result is independent of the local time zone of the machine running the code.²

Listing 2: JavaScript implementation using the built-in `Date` object

```
function findMatchingYears(targetWeekday, month, day, startYear, endYear) {
    let matchingYears = [];

    for (let year = startYear; year <= endYear; year++) {
        // JavaScript months are 0-based. UTC methods avoid local time-zone effects.
        let date = new Date(Date.UTC(year, month - 1, day));

        // getUTCDay() returns 0 for Sunday, 1 for Monday, ..., 6 for Saturday.
        if (date.getUTCDay() === targetWeekday) {
            matchingYears.push(year);
        }
    }

    return matchingYears;
}

// Example Usage: Find years where March 11 is a Saturday
// Since getUTCDay() returns 6 for Saturday, pass 6 as the targetWeekday.
console.log("Years where March 11 is a Saturday:",
    findMatchingYears(6, 3, 11, 2001, 2100));

// Output: [2006, 2017, 2023, 2028, 2034, 2045, 2051, 2056, 2062, 2073, 2079, 2084, 2090]
```

- Relies on the engine's internal Gregorian algorithm.
- Requires only one line of code to obtain the weekday.
- Iterates over a range and collects matching years.

Zeller's modular arithmetic is well suited to direct offline scans, whereas the JavaScript approach is convenient for browser-based date lookups.

6 Probabilistic Analysis, Applications, and Conclusion

Foundations: Probability Tools for Fixed Date–Time Recurrences

The probability calculation is a finite counting problem. Over one Gregorian 400-year cycle, let

$$D = 146\,097$$

be the number of days and let

$$N = D \times 86\,400 = 12\,622\,780\,800$$

be the number of second-level states.

Scope of the model. The analysis treats the Gregorian calendar as a deterministic periodic system and assumes each civil day contributes exactly 86 400 second-level states. Time zones, daylight-saving adjustments, leap seconds, and jurisdiction-specific adoption dates are not part of the state space; including them would change the timekeeping convention rather than the Gregorian recurrence counts being studied.

A random glance is modeled as a uniformly selected element of this finite sample space. If A is the event that the frozen display matches reality, then

$$\Pr(A) = \frac{|A|}{N}.$$

If repeated glances are modeled as independent draws from the same distribution, then the waiting time T until the first match is geometric:

$$\Pr(T = t) = (1 - p)^{t-1}p, \quad E[T] = \frac{1}{p}, \quad \text{Var}[T] = \frac{1 - p}{p^2}.$$

This geometric interpretation is a model for independent random inspections. It is not the same as checking a watch continuously or at fixed deterministic intervals, where successive observations are strongly dependent.⁵

Time-Only Watches (12-Hour Dial)

Let the frozen dial read $h:m:s$. A 12-hour dial without an AM/PM indicator agrees with clock time twice per 24-hour day.

Display precision	Successes per 24-hour day	p
Hours	2	$2/24 = 1/12$
Hours + Minutes	2	$2/(24 \times 60) = 1/720$
Hours + Min + Sec	2	$2/(24 \times 60 \times 60) = 1/43\,200$

The factor “2” is the familiar AM/PM duplication. A 24-hour dial, or a display that explicitly distinguishes AM from PM, halves each probability.

Adding the Calendar Fields

The sample space remains the set of N second-level states in the 400-year Gregorian cycle. Adding calendar fields changes the success event, not the underlying sample space. The month and day specify a month–date pair; the weekday is then determined by the actual calendar date. Thus a display such as

Saturday, 11 March, 10:08:00

matches only in years for which 11 March is a Saturday, and then only at the two daily instants corresponding to 10:08:00 on a 12-hour dial.

Probability of the Sample Display

The Omega Speedmaster in Fig. 1 is frozen at

Saturday, 11 March, 10:08:00 on a 12-hour dial.

400-year theoretical cycle.

- **Weekday–date matches.** Zeller’s congruence shows that 11 March is a Saturday in 56 of the 400 years.⁷
- **Strict match (time included).** A 12-hour dial has two matching seconds on each qualifying date, so

$$s_{\text{time}} = 56 \times 2 = 112.$$

- **Calendar-only match (time ignored).** Ignoring the time of day gives

$$s_{\text{cal}} = 56 \times 86\,400 = 4\,838\,400$$

second-level states.

Therefore

$$p_{\text{strict}} = \frac{112}{12\,622\,780\,800} = \frac{1}{112\,703\,400} \approx 8.87 \times 10^{-9},$$

while

$$p_{\text{calendar}} = \frac{4\,838\,400}{12\,622\,780\,800} = \frac{56}{146\,097} \approx 3.83 \times 10^{-4} \approx \frac{1}{2609}.$$

Under the independent-random-glance model, the expected number of glances is $E[T_{\text{strict}}] = 112\,703\,400$ for the strict match and $E[T_{\text{calendar}}] \approx 2609$ for the calendar-only match.

Gregorian interval 1583–2024 (empirical). Treating the Gregorian calendar as the governing calendar throughout the interval from 1 January 1583 through 31 December 2024, there are 442 complete years and 161 438 days, so

$$N_{\text{emp}} = 86\,400 \times 161\,438 = 13\,948\,243\,200.$$

In this interval,

$$\#(\text{Saturday, 11 March}) = 62,$$

so

$$s_{\text{time, emp}} = 124, \quad s_{\text{cal, emp}} = 62 \times 86\,400.$$

Hence

$$p_{\text{strict, emp}} = \frac{124}{13\,948\,243\,200} \approx 8.89 \times 10^{-9},$$

and

$$p_{\text{calendar, emp}} = \frac{62}{161\,438} \approx 3.84 \times 10^{-4}.$$

The empirical values align closely with the 400-year averages, as expected.

Most and Least Common (Weekday, Month, Date) Triples

Let a valid calendar triple mean a specified weekday, month, and date, with no time of day attached. The probability below is therefore a per-day probability over the $D = 146\,097$ days of the 400-year cycle.

Ideal 400-year cycle. For ordinary month–date pairs other than 29 February, each date occurs 400 times. Although the full 400-year Gregorian cycle contains an integer number of weeks, a fixed non-leap month–date occurs 400 times, and those 400 occurrences are distributed unevenly across the seven weekdays because of the placement of leap years. Across the Gregorian cycle, a non-leap-day month–date pair can therefore occur 56, 57, or 58 times on a given weekday.

For 29 February, the date occurs only in the 97 leap years of the cycle. Its weekday counts are 13, 14, or 15, depending on the weekday.

Thus the largest and smallest positive triple probabilities are

$$\boxed{p_{\text{max}} = \frac{58}{146\,097}}, \quad \boxed{p_{\text{min}} = \frac{13}{146\,097}}.$$

If a fixed second on a 12-hour dial is also included, these become

$$\frac{2 \times 58}{12\,622\,780\,800} \quad \text{and} \quad \frac{2 \times 13}{12\,622\,780\,800},$$

respectively. Triples with impossible dates, such as 31 November, occur zero times and are excluded from the positive minimum.

Gregorian interval 1583–2024. With 442 complete years and 108 leap days in the same Gregorian interval,

$$\#_{\max} = 65, \quad \#_{\min} = 14.$$

Therefore

$$p_{\max, \text{emp}} = \frac{65}{161\,438}, \quad p_{\min, \text{emp}} = \frac{14}{161\,438}.$$

The finite historical interval slightly changes the counts, but not the qualitative ranking: ordinary dates produce the most common triples, while 29 February produces the rarest valid triples.

Applications and Implications

The preceding calculations have modest but concrete applications. First, the 400-year recurrence counts give exact benchmark values for calendar software. Since the Gregorian cycle contains 146 097 days and repeats modulo 7, a correct implementation of Gregorian weekday arithmetic should reproduce the same frequencies for fixed month–date–weekday triples, including 56 Saturday–11 March occurrences in the cycle. Disagreement would indicate an error in leap-year handling, century-year treatment, month indexing, or weekday numbering.^{1;6}

Second, the comparison between Zeller’s congruence and the ECMAScript `Date` object illustrates algorithmic cross-validation. Zeller’s formula is an explicit modular arithmetic method; the JavaScript implementation delegates calendar arithmetic to a standardized date-time object. Agreement between independent implementations is a useful check on off-by-one errors and time-zone artifacts, especially when UTC methods are used for browser-based computations.^{7;2}

Finally, the watch problem is a compact example of finite-state probability modeling. The physical display creates an event A inside a deterministic periodic state space Ω ; the probability is obtained by counting favorable states rather than assuming independence among calendar fields. This is the same modeling discipline used in finite probability problems more generally: define the sample space, specify the sampling rule, identify the event, and compute its probability from the relevant count.⁵ The main implication is therefore methodological rather than technological: irregular deterministic structures can still yield exact probabilities when the state space is finite and the recurrence law is known.

Conclusion

The broken-watch question is ultimately a finite-state probability problem. The Gregorian calendar supplies the state space; Zeller’s congruence supplies an efficient weekday test; and uniform counting supplies the probability. The key point is that the calendar fields are not independent decorations on the watch display. Once month and date are fixed, the weekday can occur only in those years where the Gregorian cycle permits that alignment.

For the damaged Omega Speedmaster display, 11 March falls on a Saturday 56 times in the 400-year Gregorian cycle. Since the watch has a 12-hour dial, the frozen time 10:08:00 matches twice on each such day. Therefore the exact strict probability of a full date-and-time coincidence under a random second-level glance is

$$\frac{112}{12\,622\,780\,800} = \frac{1}{112\,703\,400},$$

about one chance in 1.13×10^8 . If the time of day is ignored and only the calendar display is checked, the probability is

$$\frac{56}{146\,097},$$

about one chance in 2609. These values answer the original question without requiring approximation beyond the final decimal presentation.

References

- [1] Nachum Dershowitz and Edward M. Reingold. *Calendrical Calculations*. Cambridge University Press, Cambridge, UK, 3rd edition, 2008. See p.45 for discussion of the Gregorian calendar reform and leap-year rules.
- [2] Ecma International. ECMAScript 2025 language specification (16th edition), 2025. URL <https://tc39.es/ecma262/2025/>. Standard.
- [3] International Organization for Standardization. ISO 8601-1:2019 — date and time – representations for information interchange, 2019. Standard.
- [4] Omega SA. Omega speedmaster date & day date chronograph 40 mm. <https://www.omegawatches.com/en-us/watch-omega-speedmaster-date-day-date-chronograph-40-mm-day-date-32213000>, n.d. Image modified for illustrative purposes. Original image available at <https://www.omegawatches.com/media/catalog/product/o/m/omega-speedmaster-date-day-date-chronograph-40-mm-day-date-32213000-55a8b0.png?w=1100>. Omega is a registered trademark of Omega SA.
- [5] Sheldon M. Ross. *Introduction to Probability Models*. Academic Press, 12 edition, 2019. ISBN 9780128143469.
- [6] United States Naval Observatory. Introduction to calendars. <https://www.usno.navy.mil/USNO/astronomical-applications/astronomical-information-center/calendar-information>, n.d. Retrieved 9 May 2022.
- [7] Christian Zeller. Kalender-formeln. *Acta Mathematica*, 9:131–136, 1886. doi: 10.1007/BF02406733.